Prathamesh Deshmukh (CS23115)

Lab-8 Assignment-1 To perform Linear Discriminant Analysis (LDA) and Principal Component Analysis (PCA) separately on a dataset, apply Logistic Regression on the transformed features, and compare the performance of LDA-based and PCA-based models.: To applay principle component analysis for dimensionality reduction and visualize the transformed feature space

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis as LDA
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

```
import pandas as pd
path="/content/drive/MyDrive/ML_dataset/Breast_Cancer_dataset - Breast_Cancer_dataset.csv" # Changed .gsheet to .csv
df=pd.read_csv(path)
df.head(5)
```

	id	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean	concave points_mea
0	842302	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.1471
1	842517	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.0701
2	84300903	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.1279
3	84348301	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.1052
4	84358402	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.1043

5 rows × 32 columns

```
X = df.drop(['id', 'diagnosis'], axis=1)
y = df['diagnosis'].map({'M': 1, 'B': 0})
X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.3, random_state=42)
```

```
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
```

```
pca = PCA(n_components=2)
X_train_pca = pca.fit_transform(X_train)
X_test_pca = pca.transform(X_test)
```

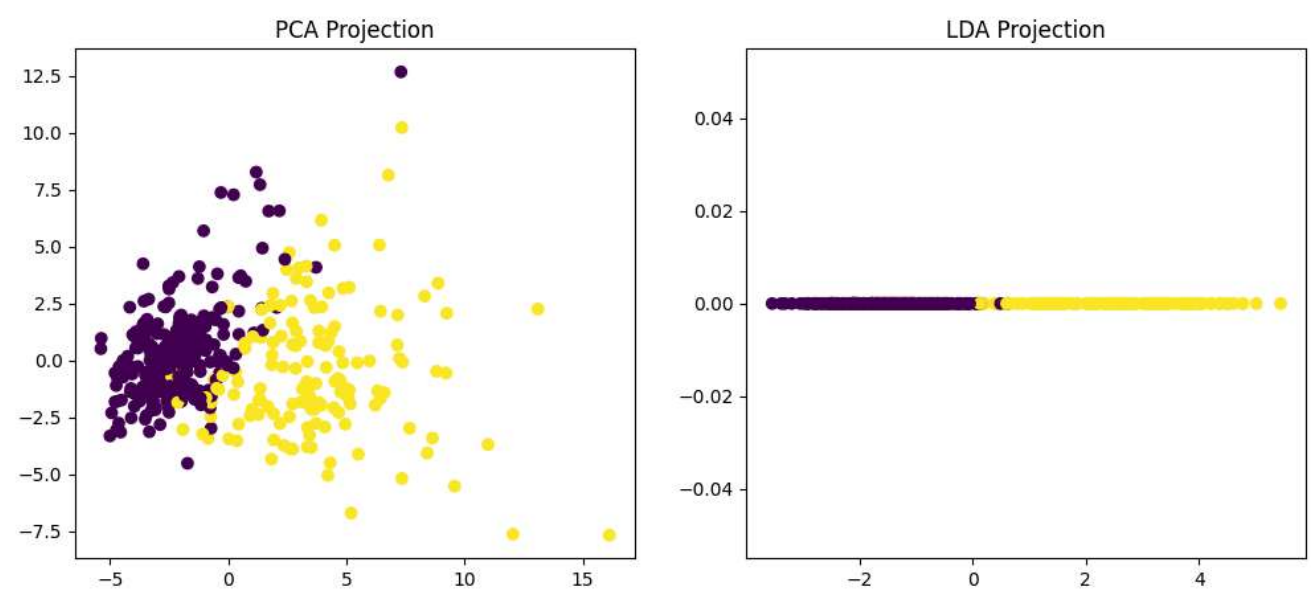
```
lda = LDA(n_components=1)
X_train_lda = lda.fit_transform(X_train, y_train)
X_test_lda = lda.transform(X_test)
```

```
clf_pca = LogisticRegression()
clf_pca.fit(X_train_pca, y_train)

clf_lda = LogisticRegression()
clf_lda.fit(X_train_lda, y_train)
```

```
▾ LogisticRegression
LogisticRegression()
```

```
y_pred_pca = clf_pca.predict(X_test_pca)
y_pred_lda = clf_lda.predict(X_test_lda)
```

```
print("Accuracy with PCA:", accuracy_score(y_test, y_pred_pca))
print("Accuracy with LDA:", accuracy_score(y_test, y_pred_lda))

print("\nClassification Report (PCA):")
print(classification_report(y_test, y_pred_pca))

print("\nClassification Report (LDA):")
print(classification_report(y_test, y_pred_lda))
```

```
Accuracy with PCA: 0.9766081871345029
Accuracy with LDA: 0.9590643274853801

Classification Report (PCA):
              precision    recall  f1-score   support

           0       0.98      0.98      0.98       108
           1       0.97      0.97      0.97        63

    accuracy                           0.98       171
   macro avg       0.97      0.97      0.97       171
weighted avg       0.98      0.98      0.98       171


Classification Report (LDA):
              precision    recall  f1-score   support

           0       0.97      0.96      0.97       108
           1       0.94      0.95      0.94        63

    accuracy                           0.96       171
   macro avg       0.95      0.96      0.96       171
weighted avg       0.96      0.96      0.96       171
```

```
plt.figure(figsize=(12,5))

plt.subplot(1,2,1)
plt.scatter(X_train_pca[:,0], X_train_pca[:,1], c=y_train)
plt.title("PCA Projection")

plt.subplot(1,2,2)
plt.scatter(X_train_lda[:,0], np.zeros_like(X_train_lda[:,0]), c=y_train) # Fixed line
plt.title("LDA Projection")

plt.show()
```